Resolution and SAT Solvers

CS1200: Introduction to Algorithms and their Limitations
Lecture 18

Fall 2026

Harvard SEAS

Put away your devices

Announcements

- Last 25 minutes today: Sender–Receiver Exercise 3 (Long Path $\leq$ SAT).
- Problem Set 6 due Thursday 11/12: resolution and ExtractAssignment are today.
- Reading: Hesterberg–Vadhan Chapter 18; Section 17.4 for the SRE.

Where we are

From modeling to solving

Last time: model problems as SAT.

- Graph k-Coloring $\leq$ CNF-SAT in time $O(n + km)$
- SAT solvers: one engine for many problems

Today: how to solve SAT.

- Resolution: decide whether φ is satisfiable
- ExtractAssignment: find a satisfying assignment
- Worst-case runtime and SAT solver performance in practice

Then: SRE 3: Modeling Long Path with SAT.

Resolution

The resolution algorithm: the big picture

1. Start with the clauses of φ, as a set $\mathcal{C}$.
2. Combine pairs of clauses into new clauses (*resolvents*); add each new one to $\mathcal{C}$. Nothing is ever removed.
3. Repeat until no new clause appears. Call the final set $\mathcal{C}_{\text{fin}}$.
4. Is the empty clause ( ) in $\mathcal{C}_{\text{fin}}$?
   - yes $\Rightarrow$ φ is unsatisfiable
   - no $\Rightarrow$ ExtractAssignment($\mathcal{C}_{\text{fin}}$) builds a satisfying assignment

Phase 1, inference: steps 1–3 decides SAT vs. UNSAT
Phase 2, commitment: step 4 builds the assignment

The resolution rule (recap)

Definition 18.1

If $\ell \in C$ and $\neg\ell \in D$: $C \diamond D = \mathrm{Simplify}\big((C - \{\ell\}) \vee (D - \{\neg\ell\})\big)$. Otherwise $C \diamond D = 1$.

Simplify a clause:

1. drop duplicate literals
2. return 1 if both x_i and $\neg x_i$ appear
3. sort the literals by variable

Example 18.3. $(x_0 \vee \neg x_1 \vee x_3 \vee \neg x_5) \diamond (x_1 \vee \neg x_4 \vee \neg x_5)$ $\quad \ell = \neg x_1, \ \neg\ell = x_1$

$= \mathrm{Simplify}(x_0 \vee x_3 \vee \neg x_5 \vee \neg x_4 \vee \neg x_5)$

$= (x_0 \vee x_3 \vee \neg x_4 \vee \neg x_5)$

Resolution: four examples

1. **Implication.** $(\neg x_0 \vee x_1) \diamond (\neg x_1 \vee x_2) = (\neg x_0 \vee x_2)$ $x_0 \Rightarrow x_1$ and $x_1 \Rightarrow x_2$, so $x_0 \Rightarrow x_2$

2. **Two complementary pairs.** $(x_0 \vee x_1 \vee \neg x_4) \diamond (\neg x_0 \vee x_2 \vee x_4)$

   - on x_0: $\mathrm{Simplify}(x_1 \vee \neg x_4 \vee x_2 \vee x_4) = 1$
   - on x_4: $\mathrm{Simplify}(x_0 \vee x_1 \vee \neg x_0 \vee x_2) = 1$

   Either way a complementary pair survives: a tautology, true under every assignment.

3. **Opposite unit clauses.** $(x_0) \diamond (\neg x_0) = ()$
   $(x_0) \wedge (\neg x_0)$ can't be satisfied, so $()$ tells us φ is unsatisfiable

4. **No complementary pair.** $(x_0 \vee x_1) \diamond (x_2 \vee \neg x_3) = 1$ adds no constraint: $\varphi \wedge 1 = \varphi$

Resolvents add no new constraints

Lemma 18.11

$\mathcal{C}$ and $\mathcal{C} \cup \{C \diamond D\}$ have the same satisfying assignments.

Why. Write $C = \ell \vee A$, $D = \neg\ell \vee B$, and let α satisfy both.

- If α makes ℓ true: D forces B.
- If α makes ℓ false: C forces A.
- Either way, α satisfies $A \vee B = C \diamond D$.

Resolvents make *implied* constraints explicit.

The Resolution Theorem

Empty clause $()$ (example 3): satisfied by no assignment.
Closed under resolution: $C \diamond D \in \mathcal{C} \cup \{1\}$ for all $C, D \in \mathcal{C}$.

Theorem 18.5

Let $\mathcal{C}$ be closed under resolution, over n variables, with clauses of width at most k. Then

1. $() \in \mathcal{C}$ if and only if $\mathcal{C}$ is unsatisfiable.
2. If $() \notin \mathcal{C}$, ExtractAssignment($\mathcal{C}$) finds a satisfying assignment in $O(n + k|\mathcal{C}|)$.

Proof of 1.

$\Rightarrow$ $() \in \mathcal{C}$: no assignment satisfies $()$, so none satisfies $\mathcal{C}$.

$\Leftarrow$ $() \notin \mathcal{C}$: ExtractAssignment constructively proves this (part 2).

Without closure: $\{(x_0), (\neg x_0)\}$ is unsatisfiable, yet $() \notin \mathcal{C}$.

In groups: close this set

ResolutionInOrder (Algorithm 18.2, corrected)

0 $C_0, \ldots, C_{m-1} \leftarrow$ clauses of φ
1 **for** $j = 1, 2, \ldots$ while C_j exists:
2 **for** $i = 0, \ldots, j-1$:
3 $R \leftarrow C_i \diamond C_j$
4 **if** $R = ()$: **return** $\bot$
5 **if** $R \neq 1$ and R is new: append R
6 **return** ExtractAssignment(all clauses)

Each pair is tried exactly once. Nothing is removed.

What's required: try every pair, including new clauses, and never loop forever.

$C_0 = (x_0 \vee x_1)$
$C_1 = (\neg x_1 \vee x_2)$
$C_2 = (\neg x_0 \vee \neg x_2)$
$C_3 = (x_1 \vee \neg x_2)$

For each pair $C_i \diamond C_j$: a new clause, a duplicate, or 1?

When does it stop? Is φ satisfiable?

In groups: close this set — answer

j	pairs	result
1	$C_0 \diamond C_1$	$(x_0 \vee x_2) =: C_4$
2	$C_0 \diamond C_2$	$(x_1 \vee \neg x_2) = C_3$, duplicate
	$C_1 \diamond C_2$	$(\neg x_0 \vee \neg x_1) =: C_5$
3	$C_0,\ C_1,\ C_2 \diamond C_3$	1, 1, 1 (none, two, none)
4	$C_0,\ C_1,\ C_2 \diamond C_4$	1, 1, 1
	$C_3 \diamond C_4$	C_0 (duplicate)
5	$C_0,\ C_1,\ C_2 \diamond C_5$	1, 1, 1
	$C_3,\ C_4 \diamond C_5$	$C_2,\ C_1$ (duplicates)

15 pairs, 2 new clauses, no ( ): closed, so **satisfiable**.

Finding the assignment

ExtractAssignment (Algorithm 18.4)

Input: closed $\mathcal{C}$ with $() \notin \mathcal{C}$

0 **for** $i = 0, \ldots, n-1$:
1 **if** $(x_i) \in \mathcal{C}$: $\alpha_i \leftarrow 1$
2 **else**: $\alpha_i \leftarrow 0$
3 $\mathcal{C} \leftarrow \mathcal{C}|_{x_i = \alpha_i}$
4 **return** α

$\mathcal{C}|_{x_i=\alpha_i}$: set x_i, then

- delete clauses made true
- delete literals made false

$\mathcal{C}$: $(x_0 \vee x_1), (\neg x_1 \vee x_2), (\neg x_0 \vee \neg x_2),$
$(x_1 \vee \neg x_2), (x_0 \vee x_2), (\neg x_0 \vee \neg x_1)$ (closed)

x_0: no $(x_0) \Rightarrow x_0 = 0$
$\mathcal{C}$: $(0 \vee x_1) \to (x_1)$, $(0 \vee x_2) \to (x_2)$,
$(\neg x_1 \vee x_2), (x_1 \vee \neg x_2)$

x_1: (x_1) present $\Rightarrow x_1 = 1$
$\mathcal{C}$: (x_2)

x_2: (x_2) present $\Rightarrow x_2 = 1$
$\mathcal{C} = \emptyset$: every clause satisfied

$\alpha = (0, 1, 1)$

Check against φ's original clauses:
$(x_0 \vee x_1), (\neg x_1 \vee x_2), (\neg x_0 \vee \neg x_2), (x_1 \vee \neg x_2)$
true literal in each: x_1, x_2, $\neg x_0$, x_1

Why closure makes the assignments safe

Not closed: $\mathcal{C} = \{(x_0 \vee x_1),\ (x_0 \vee \neg x_1)\}$

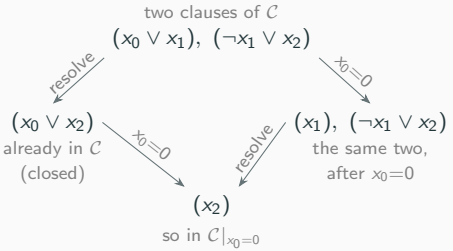
- x_0 has no unit clause, so x_0 is assigned 0 $\Rightarrow \mathcal{C}$: $(x_1),\ (\neg x_1)$
- x_1 cannot be assigned 1 or 0

Closing first adds $(x_0 \vee x_1) \diamond (x_0 \vee \neg x_1) = (x_0)$, which forces $x_0 = 1$.

If $\mathcal{C}$ is closed and $(\,) \notin \mathcal{C}$, then at every step:

(a) setting x_i cannot create $(\,)$: only a unit clause on x_i with the opposite sign could become empty, and $\mathcal{C}$ cannot contain both (x_i) and $(\neg x_i)$;

(b) $\mathcal{C}|_{x_i=\alpha_i}$ is still closed (Lemma 18.16).



So $(\,)$ never appears, and after n steps every clause is satisfied.

Runtime

Worst-Case Runtime

phase	time	why
Resolution (close the set)	$O(k_{\text{fin}} \lvert\mathcal{C}_{\text{fin}}\rvert^2)$	each *pair* tried once, $O(k_{\text{fin}})$ each
ExtractAssignment	$O(n + k_{\text{fin}} \lvert\mathcal{C}_{\text{fin}}\rvert)$	each clause shrinks at most k_{fin} times
Total	$O(n + k_{\text{fin}} \lvert\mathcal{C}_{\text{fin}}\rvert^2)$	

How big can $\mathcal{C}_{\text{fin}}$ get? Each variable: x_i, $\neg x_i$, or absent.

- At most 3^n distinct clauses; $k_{\text{fin}} \leq n$.
- Worst case $O(n \cdot 9^n)$.
- Compare this to exhaustive search of all 2^n assignments, which is worst case $O(2^n \times \text{input size})$.

When resolution is fast: 2-SAT

k-**SAT:** every clause has width (number of literals) at most k.

Resolving widths a and b gives width at most $a + b - 2$.

$k = 2$**:** $2 + 2 - 2 = 2$

- widths never grow
- $|\mathcal{C}_{\text{fin}}| = O(n^2)$ clauses of width ≤ 2
- resolution: $O(2 \cdot (n^2)^2) = O(n^4)$

$k = 3$**:** $3 + 3 - 2 = 4$

- widths can grow; back to 3^n
- no polynomial-time algorithm known for 3-SAT (Chapter 20)

Both bounds depend on the worst-case size of $\mathcal{C}_{\text{fin}}$. For $k = 2$, $|\mathcal{C}_{\text{fin}}|$ is polynomial.

SAT solvers in practice

SAT solvers: exponential worst case, fast in practice

What SAT solvers do

- Assign variables one at a time, as in ExtractAssignment, simplifying after each. Whenever a unit clause appears, set its variable at once, before choosing any other.
- If an assignment produces ( ): undo it and try the other value. Use resolution to add a clause that rules out that combination, so the search never repeats it.

3-coloring test graphs colored within 1 second

graphs		exhaustive $O(m3^n)$	maximal indep. sets + BFS (PS5) $O(1.45^n)$	SAT solver
lines of rings	(n = 1,500–3,000)	0 of 6	6 of 6	6 of 6
random clusters	(n = 40–108)	0 of 18	14 of 18	18 of 18
hard instances	(n = 20–198,000)	0 of 16	2 of 16	6 of 16

At real-world scale: in the 2016 FCC spectrum auction, coloring instances with $k = 23$ colors and n in the thousands were solved by SAT solvers, about 99% within a minute.

SRE 3

Sender–Receiver Exercise 3: Long Path $\leq$ SAT

Theorem 17.9. Long Path on a digraph with n vertices, m edges, and a path length k reduces to SAT in time $O(n^2 k)$.

- **Pair up:** one Sender, one Receiver. No partner? Raise your hand; a group of three is fine.
- **Sender:** explain the reduction top-down: the variables first, then the clauses, then why it is correct. Avoid writing; one diagram to get started is fine.
- **Receiver:** take notes and draw pictures, ask questions, and fill in as many formal details as you can.
- **Stuck?** Raise your hand; staff will come.
- **At the end:** fill out the reflection survey.